

# User Guide: Numerical Differentiation Calculator v1.0

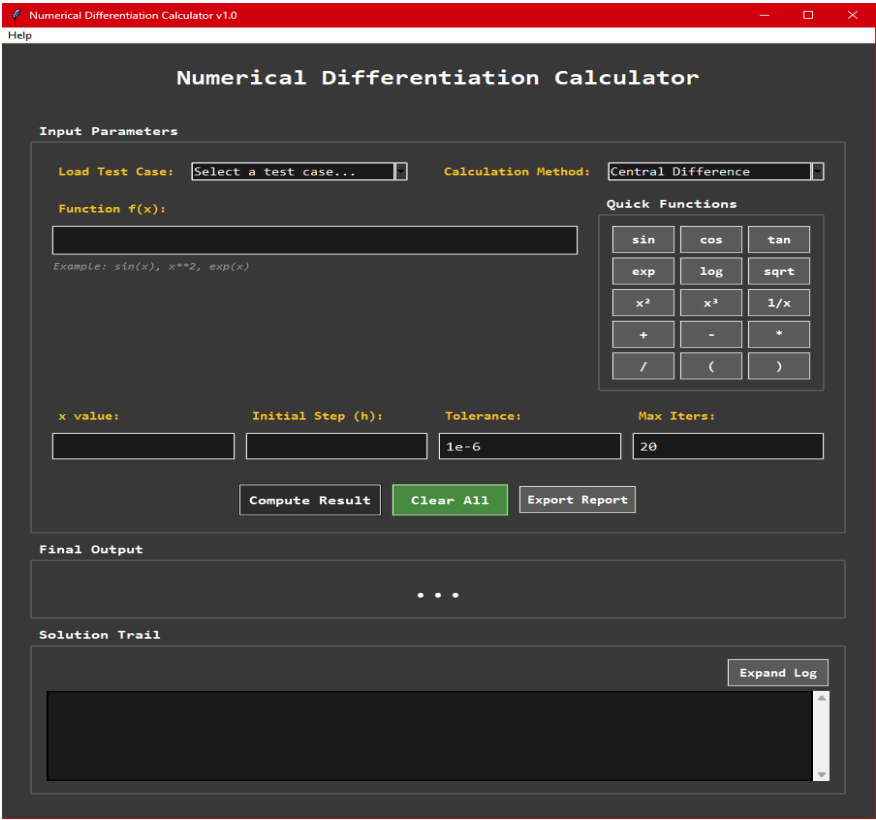
## 1. Introduction

Welcome to the Numerical Differentiation Calculator. This application is a scientific tool designed to compute the derivative of mathematical functions using finite difference methods (Central, Forward, and Backward). It features an iterative step-halving algorithm to ensure high precision and generates a comprehensive auditing trail for academic verification.

## 2. Interface Overview

When you launch the application, you will see a dark-themed, scientific calculator interface divided into three main sections:

- Input Parameters (Top):** Where you define your function, mathematical methods, and stopping rules.
- Quick Functions (Middle Right):** A clickable keypad to easily insert complex mathematical operations like  $\sin(x)$  or  $\exp(x)$ .
- Output Panels (Bottom):** Displays the final computed derivative and the step-by-step solution trail.



### 3. Step-by-Step: Performing a Calculation

#### Step 1: Enter Your Function

Click inside the **Function  $f(x)$**  text box. You can type your mathematical expression directly using your keyboard, or use the **Quick Functions** pad to insert terms.

- *Note: Use standard Python mathematical syntax (e.g.,  $x^2$  for  $x$ -squared,  $\text{sqrt}(x)$  for square root).*

#### Step 2: Set Your Mathematical Parameters

Fill in the four parameter boxes located below the function input:

- **x value:** The specific point where you want to calculate the slope (derivative).
- **Initial Step (h):** The starting distance for the finite difference calculation (e.g., 0.1).
- **Tolerance:** The target precision. The calculator will stop when the difference between iterations is smaller than this number (default is  $1e-6$ ).
- **Max Iters:** A safety limit to prevent infinite loops (default is 20).

#### Step 3: Select Your Method

Use the **Calculation Method** dropdown to choose the algorithmic approach:

- Central Difference (Default - generally the most accurate)
- Forward Difference
- Backward Difference

**[Insert Screenshot 2 Here: A cropped screenshot showing the "Input Parameters" frame filled out with a function like  $\sin(x)$ , an  $x$  value, and the Central Difference method selected.]**

#### Step 4: Compute

Click the blue **Compute Result** button.

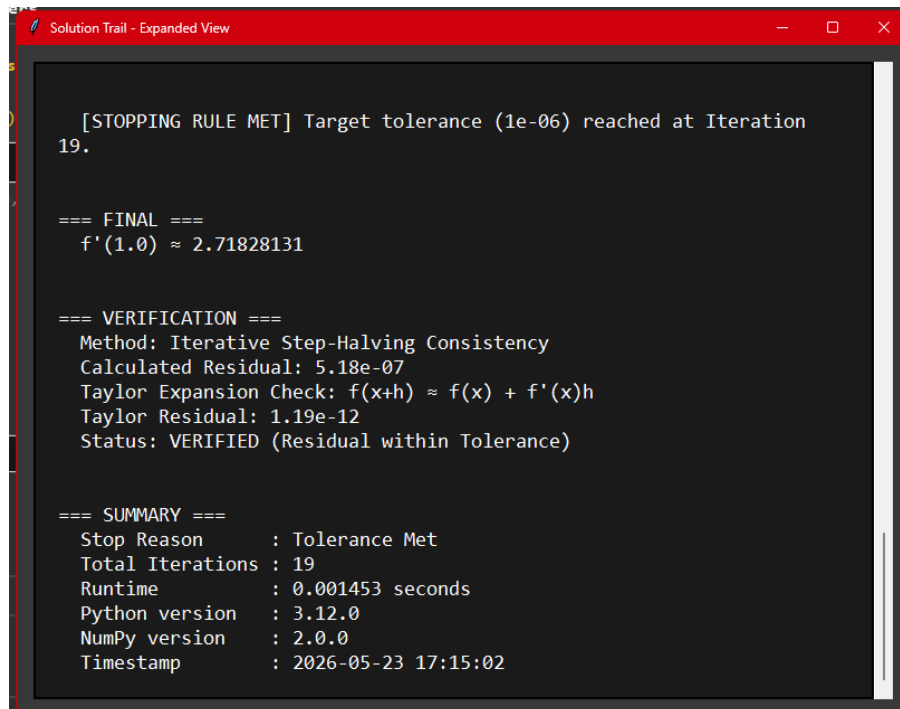
- The exact numerical derivative will appear in the large text on the bottom left.
- The right-side text box will immediately populate with the **Solution Trail**.

### 4. Auditing and Verification

One of the core features of this application is full computational transparency.

Inside the **Solution Trail** box, you can scroll through the entire iterative process. The application utilizes a dual-verify system:

1. **Iterative Consistency Check:** Verifies that the step-halving algorithm successfully converged within your requested tolerance.
2. **Taylor Expansion Check:** Back-checks the final derivative by plugging it into the local linear approximation formula, ensuring the result mathematically aligns with the original function.



```
[STOPPING RULE MET] Target tolerance (1e-06) reached at Iteration 19.

=== FINAL ===
f'(1.0) ≈ 2.71828131

=== VERIFICATION ===
Method: Iterative Step-Halving Consistency
Calculated Residual: 5.18e-07
Taylor Expansion Check: f(x+h) ≈ f(x) + f'(x)h
Taylor Residual: 1.19e-12
Status: VERIFIED (Residual within Tolerance)

=== SUMMARY ===
Stop Reason      : Tolerance Met
Total Iterations : 19
Runtime          : 0.001453 seconds
Python version   : 3.12.0
NumPy version    : 2.0.0
Timestamp        : 2026-05-23 17:15:02
```

## 5. Advanced Features

### Exporting Reports

For documentation or auditing purposes, you can save your calculation. After computing a result, click the **Export Report** button. This will open a standard file dialog allowing you to save the complete Solution Trail as a .txt file to your computer.

### Expanded Log Viewer

If you are reviewing a complex calculation with dozens of iterations, the standard UI box might feel cramped. Click the **Expand Log** button (located above the trail text box) to pop the entire trail out into a dedicated, full-screen reading window.

### Quick Load Presets

If you need to quickly test the system, use the **Load Test Case** dropdown at the very top of the window. Selecting a preset (e.g., "Test 1:  $\sin(x)$  via Central") will automatically fill out every input field with a known, verified mathematical scenario.

**[Insert Screenshot 4 Here: A screenshot showing the Expanded Log window open over the main application, demonstrating the pop-out feature.]**

## 6. Troubleshooting

- **"ERR: Missing Parameters":** Ensure you have filled out the Function,  $x$ ,  $h$ , Tolerance, and Max Iters fields. None can be left blank.
- **"ERR: Division by Zero":** Your Initial Step ( $h$ ) cannot be exactly 0.
- **Mathematical Domain Errors:** If you see an error regarding a "math domain," you are attempting to calculate an impossible value (such as the square root of a negative number, or dividing by zero). Check your  $x$  value against your  $f(x)$  function.